Báo cáo tuần

I. YOLOv3 Loss Function

1. Tổng quan

Hàm mất mát trong mô hình YOLOv3 được thiết kế nhằm tối ưu đồng thời ba nhiệm vụ chính:

- **Localization** (xác định bounding box)
- **Objectness** (xác suất có object)
- **Classification** (phân loại object)

Tổng loss được định nghĩa như sau:

$$\mathcal{L} = \lambda_{box} \cdot \mathcal{L}_{box} + \mathcal{L}_{obj} + \lambda_{noobj} \cdot \mathcal{L}_{noobj} + \mathcal{L}_{class}$$

Trong đó:

- $\lambda_{box} = 5.0$
- $\lambda_{noobj} = 0.5$

2. Bounding Box Loss (Localization Loss)

Bounding box được biểu diễn dưới dạng:

$$(x, y, w, h)$$

Trong đó:

- x, y: offset trong cell
- w, h: scale theo anchor

Mô hình dự đoán:

$$\hat{x}, \hat{y} = \sigma(t_x), \sigma(t_y)$$
$$\hat{w}, \hat{h} = e^{t_w}, e^{t_h}$$

Loss được tính bằng **Mean Squared Error (MSE)**:

$$\mathcal{L}_{box} = \sum_{obj} \left[(x - \hat{x})^2 + (y - \hat{y})^2 + (w - \hat{w})^2 + (h - \hat{h})^2 \right]$$

3. Objectness Loss

Objectness biểu diễn xác suất một bounding box chứa object.

Trong implementation này, thay vì dùng nhãn nhị phân (0/1), ta sử dụng **IoU làm soft target**:

$$\mathcal{L}_{obj} = \sum_{obj} BCE(\hat{p}_{obj}, IoU)$$

Điều này giúp mô hình học được:

confidence phản ánh chất lượng bounding box

4. No-object Loss

Đối với các anchor không chứa object:

$$\mathcal{L}_{noobj} = \sum_{noobj} BCE(\hat{p}_{obj}, 0)$$

Do số lượng negative rất lớn, loss này được giảm trọng số bằng:

$$\lambda_{noobj} = 0.5$$

5. Classification Loss

YOLOv3 sử dụng **multi-label classification**, do đó không dùng softmax mà dùng sigmoid cho từng class độc lập.

Loss được tính bằng:

$$\mathcal{L}_{class} = \sum_{obj} BCE(\hat{p}_{class}, y_{onehot})$$

Trong đó:

- y_{onehot} là vector one-hot của class

- áp dụng chỉ trên các anchor có object

6. Tổng hợp Loss

Tổng loss cuối cùng:

$$\mathcal{L} = 5 \cdot \mathcal{L}_{box} + \mathcal{L}_{obj} + 0.5 \cdot \mathcal{L}_{noobj} + \mathcal{L}_{class}$$

II. Xây dựng target tensor trong dataset

Lý do xây target trong Dataset

- Tận dụng Multi-worker của CPU:

  + PyTorch DataLoader cho phép:

  + DataLoader(..., num_workers=N)

- Mỗi worker:

  - load ảnh
  - augment
  - xây target tensor song song

- Tăng tốc huấn luyện

  - CPU xử lý preprocessing
  - GPU chỉ tập trung forward/backward

- Nếu xây target trong training loop thì GPU phải chờ CPU

III. Thay thế Backbone Darket-53 bằng ResNet-50 Pretrained

Trong kiến trúc YOLOv3 gốc, backbone được sử dụng là Darknet-53, một mạng CNN sâu gồm 53 lớp, được thiết kế đặc biệt cho bài toán object detection.

Tuy nhiên, trong hệ thống này, backbone Darknet-53 được thay thế bằng ResNet-50 với trọng số pretrained trên ImageNet nhằm cải thiện hiệu suất huấn luyện và độ chính xác của mô hình.

Hạn chế của Darknet-53

Việc sử dụng Darknet-53 trong YOLOv3 truyền thống tồn tại một số hạn chế:

- Train from scratch: Không tận dụng được tri thức từ dataset lớn như ImageNet
- Thời gian hội tụ lâu: Cần nhiều epoch để đạt hiệu quả tốt

- Khó tối ưu
- Yêu cầu tài nguyên lớn khi training từ đầu

Sử dụng ResNet-50 Backbone

Lý do lựa chọn ResNet-50

- Được pretrain trên ImageNet
- Kiến trúc residual giúp gradient flow tốt
- Nhẹ hơn và phổ biến trong nhiều ứng dụng

Backbone được xây dựng dựa trên ResNet-50 và trích xuất đặc trưng ở nhiều mức:

Input
→ Stem (conv1 + bn + relu + maxpool)
→ layer1
→ layer2 → route1 (C3)
→ layer3 → route2 (C4)
→ layer4 → route3 (C5)

Sau đó sử dụng các lớp 1×1 convolution để giảm số kênh để phù hợp kích thước với DetectonBlock:

Feature map	Original channels	Reduced
route1	512	256
route2	1024	512
route3	2048	1024

Lợi ích của Pretrained Backbone

1. **Tăng tốc hội tụ**
   - Trọng số pretrained đã học giúp mô hình: hội tụ nhanh hơn đáng kể so với train từ đầu
2. **Cải thiện độ chính xác**
   - Feature extractor tốt hơn
3. **Giảm chi phí huấn luyện**

- Ít epoch hơn
- Ít tune hơn